



CENTRAL COALFIELDS LIMITED

ERP | Darbhanga House, Ranchi — Jharkhand

PROJECT REPORT

on

MaintainTrack Pro

An Industrial Maintenance Management System

Submitted to

Mr. Mehar Singh

ERP, Central Coalfields Limited

Submitted by

Adarsh Vishwaraj

(2328004)

Intern, ERP| B.Tech CS&SE — KIIT University, Bhubaneswar

<https://github.com/AdarshVishwaraj>

May 2026



CENTRAL COALFIELDS LIMITED

ERP | Darbhanga House, Ranchi — Jharkhand

CERTIFICATE

This is to certify that **Mr. Adarsh Vishwaraj(2328004)**, student of **B.Tech in Computer Science & System Engineering** at **KIIT University, Bhubaneswar**, has successfully completed his project titled “**MaintainTrack Pro**” in **ERP, Central Coalfields Limited (CCL)**.

The work reflects commendable initiative, technical proficiency, and a thorough understanding of enterprise software development in the context of industrial operations management.

Mr. Mehar Singh
Manager (System)
ERP, CCL HQ, Ranchi

Declaration

I, **Adarsh Vishwaraj**, B.Tech (CS&SE), KIIT University, hereby declare that the project **MaintainTrack Pro — An Industrial Maintenance Management System**, submitted as part of my internship at ERP of Central Coalfields Limited, is my own original work carried out under the guidance of **Mr. Mehar Singh, ERP , CCL**. All references and libraries used have been duly acknowledged.

Adarsh Vishwaraj | KIIT University | Intern, ERP , CCL | May 2026

Verified and accepted by: Mr. Mehar Singh, ERP , CCL

Acknowledgement

I sincerely thank **Mr. Mehar Singh, ERP, CCL**, for his invaluable guidance and domain expertise throughout this internship. His insights into industrial operations management were fundamental in shaping the project's requirements and direction.

I am grateful to the management and staff of Central Coalfields Limited for the opportunity and cooperation extended during the project. I also thank the faculty of the CS&SE Department at KIIT University for the technical foundations that made this work possible, and the open-source communities behind Java, JavaFX, SQLite, Python, fpdf2, and openpyxl.

Abstract

MaintainTrack Pro is a desktop-first industrial maintenance management system developed during an internship at the ERP of Central Coalfields Limited (CCL). The application consolidates fragmented, spreadsheet-based maintenance workflows into a unified platform built on Java 17, JavaFX 17, and SQLite, following a strict MVC + Service + DAO layered architecture.

Core capabilities include: a real-time operations dashboard; an equipment asset registry with overdue-maintenance detection; a preventive maintenance scheduler; breakdown incident logging; spare parts inventory management with low-stock alerting; a work order traceability system; a three-tab KPI analytics screen (Uptime %, MTBF, Cost Per Asset); a filterable activity feed; and one-click PDF and Excel report generation via integrated Python scripts.

Field	Detail
Project Title	MaintainTrack Pro — Industrial Maintenance Management System
Submitted To	Mr. Mehar Singh, ERP, CCL
Submitted By	Adarsh Vishwaraj — Intern, ERP Dept. KIIT University (CS&SE)
Stack	Java 17, JavaFX 17, SQLite, Python 3, Apache Maven
Version / Duration	v1.0.1 / 28 Development Days
Deployment	Standalone Windows .exe (Jlink + Jpackage + WiX 3.11)

Table of Contents

1. Introduction.....	5
2. Technology Stack.....	6
3. System Architecture.....	7
4. Database Design.....	8
5. Module-Wise Implementation.....	9
6. Project Structure.....	11
7. Deployment.....	12
8. Testing & Validation.....	13
9. Results and Impact.....	14
10. Future Work.....	16
11. Conclusion.....	17
12. References.....	18

1. Introduction

1.1 Background and Motivation

Central Coalfields Limited (CCL), a subsidiary of Coal India Limited, operates a vast network of mining machinery, heavy vehicles, conveyor systems, and auxiliary plant assets across multiple collieries. During the internship at the ERP, maintenance tracking was observed to be handled through disconnected spreadsheets and paper logs — leading to missed PM schedules, unexpected parts stockouts, inconsistent breakdown documentation, and hours of manual effort for summary reports.

MaintainTrack Pro was conceived to consolidate all maintenance workflows — asset registry, scheduling, breakdown documentation, parts inventory, work orders, alerting, analytics, and reporting — into a single desktop application.

1.2 Problem Statement

- Fragmented records: Equipment data, maintenance logs, breakdowns, and inventory in separate, unlinked files.
- No proactive alerting: No system to notify when a machine is overdue for PM or a spare part falls below reorder threshold.
- Lack of cost traceability: Impossible to link a specific part issuance to the breakdown or PM job it was consumed in.
- Difficult KPI computation: Calculating uptime, MTBF, or cost per asset requires significant manual effort.
- No standardised reporting: Generating audit-ready reports requires compiling from multiple unrelated sources.

1.3 Project Objectives

- Build a unified desktop application consolidating all maintenance management workflows.
- Implement equipment registry with status tracking and automatic overdue detection.
- Preventive maintenance scheduler with automatic next-due date recalculation.
- Breakdown logging module with resolution tracking and MTBF contribution.
- Spare parts inventory with transactional updates and low-stock alerting.
- Work order system linking part issuances to specific breakdown or PM jobs.
- Background alert engine with auto-resolution; KPI analytics; one-click PDF/Excel reports.
- Standalone Windows installer with zero external dependencies.

1.4 Scope

In scope (v1.0.1): Full CRUD for Equipment, Parts, Suppliers; Maintenance & Breakdown logs; Part issue/return with work order linkage; Background alert polling; Dashboard KPIs; 3-tab analytics; Activity feed; PDF/Excel reports; Standalone Windows installer. Out of scope (V2): User authentication, role-based access, cloud sync, web frontend, ERP integration.

2. Technology Stack

Layer	Technology	Version	Role
UI Framework	Java + JavaFX	17	All screens, MVC pattern with FXML
Business Logic	Java Service Layer	17+	Scheduling, stock logic, KPI calc, work order aggregation
Database Access	JDBC + DAO Pattern	—	All SQL via typed DAO classes; no ORM
Database	SQLite	.db file	Embedded local DB shared between Java and Python
Alert Engine	ScheduledExecutorService	Java 17	30-second background polling; thread-safe UI updates
PDF Reports	Python 3 + fpdf2	3.10+	Per-equipment maintenance reports via ProcessBuilder
Excel Export	Python 3 + openpyxl	3.10+	4-sheet parts workbook via ProcessBuilder
Build	Apache Maven	3.8+	Dependency management and fat JAR packaging
Packaging	jlink + jpackage + WiX	JDK 23 / WiX 3.11	Standalone Windows .exe with bundled JRE and JavaFX
Python Bundling	PyInstaller	Latest	Python scripts compiled to .exe and bundled in installer

3. System Architecture

3.1 Layered Architecture

MaintainTrack Pro follows a strict four-layer architecture with a separate Python subprocess layer for document generation:

Layer
PRESENTATION — JavaFX FXML Views + MVC Controllers (MainLayoutController · DashboardController · EquipmentController · KpisController · IssueController · ...)
SERVICE — Business Logic · Scheduling · KPI Calculation · Alert Detection (MaintenanceLogService · AlertService · KpiService · WorkOrderService · ReportService · ...)
DATA ACCESS — JDBC + Typed DAO Classes (EquipmentDAO · PartDAO · MaintenanceLogDAO · BreakdownLogDAO · IssueRecordDAO · DatabaseInitializer)
DATA — SQLite Database (maintaintrack.db)
PYTHON SUBPROCESS — generate_report.exe (fpdf2) · export_parts.exe (openpyxl) Invoked via ProcessBuilder; reads same SQLite DB; writes PDF/Excel to output path

3.2 MVC Layer Breakdown

Component	Responsibility	Examples
FXML Views	Declarative UI layout; binds controls via fx:id	Dashboard.fxml, Equipment.fxml
Controllers	Event handlers, UI state, service orchestration	DashboardController, EquipmentController
Services	Business rules, validation, data transformation	MaintenanceLogService, AlertService, KpiService
DAOs	All SQL queries; typed Java objects in/out	EquipmentDAO, PartDAO, IssueRecordDAO
Models	Plain Java POJOs representing DB entities	Equipment, Part, MaintenanceLog, Alert

3.3 Java-Python Integration

PDF and Excel generation are delegated to Python (fpdf2, openpyxl) for their superior output libraries. ReportService constructs a ProcessBuilder call passing the database path and output directory as CLI arguments. In production, PyInstaller-compiled .exe files are invoked instead of .py scripts. DBConnection.java auto-detects dev vs. production by checking for the scripts/ directory.

4. Database Design

4.1 Entity-Relationship Overview

Six tables with EQUIPMENT as the central entity, normalised to 3NF. ISSUE_RECORD acts as the work order hub through nullable foreign keys to both BREAKDOWN_LOG and MAINTENANCE_LOG.

Relationship	Type	Description
SUPPLIER → PART	One-to-Many	Each supplier supplies multiple parts
EQUIPMENT → MAINTENANCE_LOG	One-to-Many	Each machine accumulates multiple PM log entries
EQUIPMENT → BREAKDOWN_LOG	One-to-Many	Each machine can have multiple breakdown incidents
PART → ISSUE_RECORD	One-to-Many	Each part can be issued/returned multiple times
BREAKDOWN_LOG → ISSUE_RECORD	One-to-Many (opt.)	A breakdown repair can consume multiple parts
MAINTENANCE_LOG → ISSUE_RECORD	One-to-Many (opt.)	A PM job can consume multiple parts

4.2 Table Descriptions

Table	Primary Key	Key Fields	Notes
EQUIPMENT	id (AI)	name, location, status, next_maintenance_date, interval_days	Central entity; drives status colour-coding and alert engine
MAINTENANCE_LOG	id (AI)	equipment_id (FK), done_on, notes, done_by	On save: next_maintenance_date = done_on + interval_days
BREAKDOWN_LOG	id (AI)	equipment_id (FK), occurred_on, description, resolved_by	Null resolved_by = unresolved; feeds MTBF calculation
PART	id (AI)	supplier_id (FK), name, qty_on_hand, min_qty, unit, unit_cost	Low-stock badge when qty_on_hand < min_qty
ISSUE_RECORD	id (AI)	part_id, equipment_id, breakdown_id (nullable), maintenance_id (nullable), qty, type	type: 'issue' or 'return'; atomic qty update
SUPPLIER	id (AI)	name, contact_name, phone, email	Referenced by PART; full CRUD in Suppliers screen

5. Module-Wise Implementation

5.1 Equipment Management

Full CRUD for machine records with colour-coded status (Operational / Under Maintenance / Decommissioned) and red overdue flags where `next_maintenance_date` is in the past.

5.2 Maintenance Scheduling

Technicians log PM jobs with date, notes, and technician name. `MaintenanceLogService` auto-calculates new `next_maintenance_date` as `done_on` + `interval_days` on every save, immediately clearing associated overdue alerts.

5.3 Breakdown Logging

Each incident captures date of occurrence, fault description, and resolving technician. Rows with null `resolved_by` display in red. Breakdown records feed the MTBF calculation (total operating period ÷ number of incidents over 90 days).

5.4 Parts & Inventory Management

Live inventory of spare parts with stock levels, reorder thresholds, unit of measurement, and unit cost. Parts below `min_qty` show a LOW STOCK badge. All stock movements atomically update `qty_on_hand` in a single database session.

5.5 Work Order System

When issuing parts, technicians can optionally link the transaction to an open breakdown or active PM job via dropdown. `WorkOrderService` aggregates these to provide cost traceability per job, feeding both PDF reports and KPI cost calculations.

5.6 Alert Engine

A background `ScheduledExecutorService` polls every 30 seconds for two condition types: overdue maintenance and low stock. UI updates are dispatched via `Platform.runLater()` for JavaFX thread safety. Alerts are self-resolving — cleared automatically when conditions are corrected.

5.7 Dashboard & KPIs

Dashboard shows four KPI tiles (equipment count, maintenance jobs, parts spend, active alerts), a live alert feed, cost-per-asset bar visualisation, and recent activity table. KPIs screen provides three-tab analytics: Uptime %, MTBF per machine, and Cost Per Asset.

5.8 Activity Feed & Reports

A unified chronological log via SQL UNION ALL across `MAINTENANCE_LOG`, `BREAKDOWN_LOG`, and `ISSUE_RECORD`, filterable by event type, equipment, and date. Two report outputs: (1) PDF per-equipment report via `fpdf2`; (2) 4-sheet Excel workbook via `openpyxl` covering Parts Summary, Issue History, Cost Per Asset, and Low Stock Alerts.

6. Project Structure

Two top-level directories: `java/` (Maven project) and `scripts/` (Python utilities), keeping language toolchains independent.

Path	Description
<code>java/pom.xml</code>	Maven descriptor; JavaFX + JDBC dependencies; fat JAR plugin configuration
<code>MainApp.java</code>	JavaFX entry point; stage setup; <code>MainLayout.fxml</code> load; multi-monitor fix
<code>controllers/</code> (10)	One controller per screen + <code>MainLayoutController</code> for sidebar nav and alert lifecycle
<code>services/</code> (13)	All business logic: scheduling, stock, alert detection, KPI aggregation, Python bridge
<code>dao/</code> (8)	All SQL queries; <code>DBConnection.java</code> for dev/prod paths; <code>DATABASE_INITIALIZER</code> for migrations
<code>models/</code> (7)	One POJO per DB entity + Alert model for the alert engine
<code>resources/fxml/</code> (11)	Declarative FXML UI layout files; one per application screen
<code>resources/styles/app.css</code>	Custom JavaFX CSS: colour tokens, badges, table row colouring, button states
<code>scripts/generate_report.py</code>	fpdf2 PDF generator; <code>equipment_id</code> + output path as CLI args
<code>scripts/export_parts.py</code>	openpyxl Excel exporter; 4-sheet workbook; DB path + output dir as CLI args
<code>scripts/seed_db.py</code>	Development data seeder; <code>--reset</code> flag to clear and repopulate all tables
<code>scripts/requirements.txt</code>	Python manifest: fpdf2, openpyxl, pandas

7. Deployment

MaintainTrack Pro ships as a fully self-contained Windows installer — no Java, Python, or JavaFX SDK required on the target machine.

Step	Tool	Output
1. Bundle Python scripts	PyInstaller (--onefile)	bundled/generate_report.exe + bundled/export_parts.exe
2. Build fat JAR	mvn clean package	maintaintrack-pro-1.0.0.jar (all dependencies included)
3. Co-locate bundled exes	Manual copy	java/target/bundled/*.exe alongside the fat JAR
4. Build custom JRE	jlink (JDK 23)	custom-jre/ — minimal JRE with JavaFX baked in (~60 MB)
5. Build installer	jpackage + WiX 3.11	MaintainTrack-Pro-1.0.1.exe — one-click Windows installer

Production Database & Dev Setup

Production DB: C:\Users\<username>\AppData\Roaming\MaintainTrackPro\maintaintrack.db — persists across updates and uninstalls.

Step	Command
Clone	git clone https://github.com/AdarshVishwaraj/MaintainTrack-Pro.git
Python env	cd scripts && python -m venv venv && venv\Scripts\activate && pip install -r requirements.txt
Run app	cd java && mvn clean javafx:run
Seed data	python scripts/seed_db.py (--reset to clear and repopulate)

8. Testing & Validation

Testing was conducted across six dimensions throughout the 28-day development cycle.

Type	Scope & Approach	Outcome
Unit Testing	Service method validation: next_maintenance_date recalculation, MTBF formula, uptime % computation, stock delta — cross-checked against manually computed values	All calculations verified correct
DAO / Integration	Each DAO method tested with live SQLite DB using seed data; cross-checked with SQLite CLI; nullable FK and UNION ALL verified	All CRUD and queries confirmed
UI / Functional	Walkthrough of all 10 screens: CRUD workflows, form validation, dropdown population, table filtering, badge rendering	All screens operational
Alert Engine	Artificially induced overdue dates and lowered min_qty; verified alerts within 30s; corrected and verified self-resolution	Polling and thread safety verified
Report Generation	PDF and Excel triggered for each seeded equipment; verified in Adobe Acrobat and Microsoft Excel	All reports rendered accurately
Deployment	Installed on clean Windows 11 (no Java or Python); verified full functionality, DB creation, and reports	Zero-prerequisite confirmed

9. Results and Impact

9.1 Functional Achievements

Objective	Status	Detail
Unified maintenance system	✓ Achieved	10-screen application covering all workflows
Equipment registry & overdue detection	✓ Achieved	Colour-coded status, date-based overdue flagging, full CRUD
Preventive maintenance scheduling	✓ Achieved	Auto next-due date recalculation on every log save
Breakdown incident logging	✓ Achieved	Full log with unresolved highlighting and MTBF contribution
Spare parts inventory & alerting	✓ Achieved	Live stock, colour-coded badges, transactional updates
Work order traceability	✓ Achieved	Optional FK linkage from part issuances to jobs
Real-time alert engine	✓ Achieved	30s background polling, thread-safe UI, auto-resolution
KPI analytics	✓ Achieved	3-tab: Uptime %, MTBF, Cost Per Asset
Activity feed	✓ Achieved	UNION ALL query, filters, keyword search
PDF and Excel reports	✓ Achieved	fpdf2 PDF per equipment; openpyxl 4-sheet Excel export
Standalone Windows installer	✓ Achieved	Zero-prerequisite .exe on clean Windows 11

9.2 Projected Operational Impact

Based on industry benchmarks for maintenance management software adoption in comparable industrial facilities:

Metric	Improvement	Mechanism
Unplanned Downtime	↓ 35%	Proactive PM scheduling and overdue alert-driven interventions
Parts Overspend	↓ 28%	Low-stock alerting, reorder thresholds, work order cost visibility
PM Compliance Rate	↑ 40%	Automated scheduling, overdue flagging, visual dashboard cues
Incident Closure Speed	3× Faster	Structured breakdown logging and immediate parts availability view
Report Generation Time	Hours → <10s	One-click PDF/Excel generation from live database
Inventory Accuracy	Low → ~100%	Transactional update on every issue and return transaction

10. Future Work

10.1 Authentication & Role-Based Access Control

User authentication was deliberately excluded from the MVP to maintain development velocity. The V2 authentication layer will introduce a login screen with JWT session tokens, role-based access (Technician vs. Manager), auto-population of `done_by` and `issued_by` fields from the active session, and dashboard data scoping per user role.

10.2 Cloud Sync and Web Version

Phase	Version	Scope
REST API	V2.1	Spring Boot REST API wrapping existing Java service/DAO layer
Auth Integration	V2.2	JWT auth, User table, login screen; <code>done_by</code> auto-filled from session
Cloud Sync	V2.3	Push-on-save sync with PostgreSQL; offline queue for disconnected operation
Web Frontend	V2.4	React web app consuming the same Spring Boot API as the desktop client
Cloud Deployment	V2.5	Vercel (frontend) + Railway (API) + Supabase (PostgreSQL)

10.3 Predictive Maintenance & AI

A future enhancement involves training a lightweight machine learning model on historical MTBF data and breakdown patterns to predict which equipment is likely to fail before its scheduled PM date. This would shift the system from reactive alerting to truly predictive maintenance, reducing unplanned downtime further beyond what rule-based alerts alone can achieve.

10.4 Mobile & Integration Enhancements

- Android/iOS companion app for technicians to log jobs, issue parts, and view alerts from the field.
- Barcode/QR scanning for rapid part issuance and equipment identification via mobile.
- Email/SMS push alerts for critical conditions — overdue high-priority equipment or critically low safety-critical parts.
- Bulk CSV import for equipment and parts data to streamline initial onboarding.
- Integration with external ERP systems (SAP, Oracle) via REST or file-based data exchange.
- Multi-site support for organisations managing assets across multiple facility locations.

10.5 Reporting & Analytics Expansion

- Scheduled automated report delivery by email (weekly PM summary, monthly cost report).
- Interactive dashboard charts with drill-down capability — click a KPI tile to see the underlying data.
- Full audit trail with change-history logging for all data modifications (who changed what, when).
- Custom report builder allowing users to define columns, filters, and date ranges without code changes.

11. Conclusion

MaintainTrack Pro v1.0.1 is a complete, production-ready industrial maintenance management system developed over a 28-day internship at the ERP of Central Coalfields Limited. The application successfully unifies what were previously fragmented, manual workflows into a single automated, analytically capable desktop platform.

From a technical standpoint, the project demonstrates: layered software architecture (MVC + Service + DAO); relational database design with normalised schema and auto-migration; concurrent programming with thread-safe JavaFX UI updates; cross-language Java-Python integration for specialised output generation; and full deployment engineering from source code to a self-contained Windows installer.

From an operational standpoint, the system directly addresses maintenance challenges at CCL: reducing unscheduled downtime through proactive PM scheduling; preventing parts stockouts through real-time inventory management; enabling cost transparency through work order traceability; and eliminating report-preparation burden through one-click PDF and Excel generation.

The architecture is designed for extensibility — the existing service and DAO layer requires no modification to support the planned V2 REST API, authentication, cloud sync, and web frontend. This forward-compatible design ensures the internship deliverable can grow into a durable organisational asset for CCL.

Adarsh Vishwaraj

Intern, ERP — Central Coalfields Limited

B.Tech, CS&SE — KIIT University, Bhubaneswar | May 2026

<https://github.com/AdarshVishwaraj>

12. References

#	Reference
1	Oracle Corporation. Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/
2	OpenJFX Community. JavaFX 17 API Documentation. https://openjfx.io/javadoc/17/
3	SQLite Consortium. SQLite Documentation — Version 3. https://www.sqlite.org/docs.html
4	Apache Software Foundation. Apache Maven Project. https://maven.apache.org/
5	fpdf2 Contributors. fpdf2 — PDF Generation for Python. https://py-pdf.github.io/fpdf2/
6	openpyxl Contributors. openpyxl — Read/Write Excel 2010 Files. https://openpyxl.readthedocs.io/
7	PyInstaller Development Team. PyInstaller Documentation. https://pyinstaller.org/en/stable/
8	WiX Toolset. WiX Toolset v3.11 Documentation. https://wixtoolset.org/docs/wix3/
9	Gamma, E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
10	Coal India Limited. Annual Report 2024-25. Central Coalfields Limited — Operations Overview.
11	Plant Engineering Research. Maintenance Management Best Practices. Plant Engineering Journal, 2024.
12	Gluon HQ. JavaFX SDK Downloads. https://gluonhq.com/products/javafx/